

Lab2

Course: ARTI 404 – Image Processing

Lab Topic: Sampling, Quantization & Image Arithmetic / Set Operations

Student Name: Muhannad Almutlaq

Student ID: 2240006060

1. Lab Outcomes

The objective of this lab is to explore fundamental digital image processing concepts at the pixel and array levels: sampling and quantization of images, overflow-safe arithmetic on 8-bit images, and set operations on grayscale images.

2. Procedural Tasks

Task 1: Sampling and Quantization

- **Description:** Defined functions to sample an image (nearest-neighbor downsampling with `cv2.resize`) and to quantize it to a given number of gray levels, plus a function that plots the original, sampled, and quantized images side by side. Applied them to a grayscale image with a sampling factor of 14 and 9 quantization levels. The 360x540 image is sampled down to 25x38 pixels, which gives the blocky look, and the quantization step of $256 // 9 = 28$ leaves 10 distinct gray values (0 to 252).
- **Code & Output:**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def sample_image(image, factor):
    height, width = image.shape[:2]
    sampled_image = cv2.resize(image, (width // factor, height // factor),
                                interpolation=cv2.INTER_NEAREST)

    return sampled_image

def quantize_image(image, levels):
    quantized_image = np.floor(image / (256 // levels)) * (256 // levels)
    quantized_image = quantized_image.astype(np.uint8)
```

```
    return quantized_image

def plot_images(original, sampled, quantized):
    plt.figure(figsize= (12, 4))

    # Original Image
    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap= "gray")
    plt.title("Original Image")
    plt.axis("off")

    # Sampled Image
    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap= "gray")
    plt.title("Sampled Image")
    plt.axis("off")

    # Quantized Image
    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap= "gray")
    plt.title("Quantized Image")
    plt.axis("off")

    plt.show()

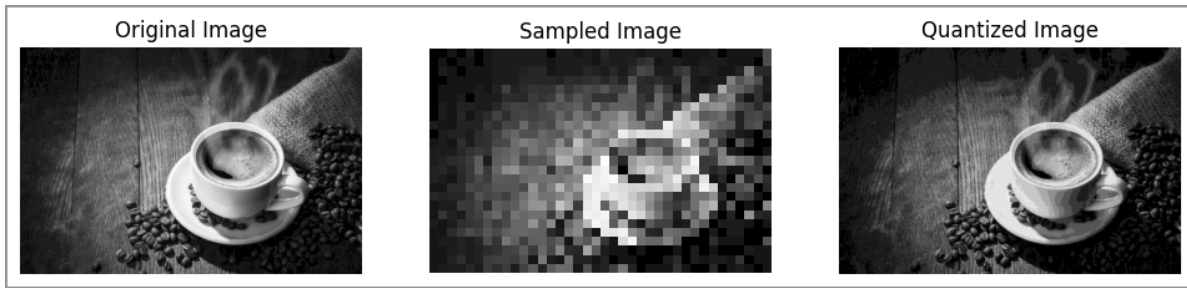
image_path= "./images/image.png"

sampling_factor= 14
quantization_levels= 9

original_image= cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if original_image is None:
    print(f"Error: Unable to load image at {image_path}")

sampled_image= sample_image(original_image, sampling_factor)
quantized_image= quantize_image(original_image, quantization_levels)

plot_images(original_image, sampled_image, quantized_image)
```



Task 2: Image Addition with Clamping

- **Description:** Resized two images to 400x400 using Lanczos resampling, converted them to uint16 so the sum cannot overflow, added them, and clamped the result to 0-255 with np.clip. Where the second image is white the sum saturates to white, so the first image only shows through the dark silhouette.
- **Code & Output:**

```
from PIL import Image

img1= Image.open('./images/task1v1.png')
img2= Image.open('./images/task1v2.png')

resize= (400, 400)
img1= img1.resize(resize, Image.Resampling.LANCZOS)
img2= img2.resize(resize, Image.Resampling.LANCZOS)

im1arr= np.asarray(img1, dtype= np.uint16)
im2arr= np.asarray(img2, dtype= np.uint16)

addition= np.clip(im1arr + im2arr, 0, 255).astype(np.uint8)

resultImage= Image.fromarray(addition)
resultImage.show()
```

img1: task1v1.png (resized)



img2: task1v2.png (resized)



Output: resultImage.show()



Task 3: Bitwise Union of Two Images

- **Description:** Displayed two color images, resized both to 400x400, and combined them with the bitwise OR operator (|) to see the overlay effect. OR sets every bit that is set in either image, so the result is a brighter, color-shifted mix of the two.
- **Code & Output:**

```
img3= Image.open('./images/task2v1.png')
img4= Image.open('./images/task2v2.png')

img3.show()
img4.show()

img3= img3.resize(resize, Image.Resampling.LANCZOS)
img4= img4.resize(resize, Image.Resampling.LANCZOS)

im3arr= np.asarray(img3)
im4arr= np.asarray(img4)

union= im4arr | im3arr

resultImage2= Image.fromarray(union)
resultImage2.show()
```

img3.show()



img4.show()



Output: resultImage2.show()



3. Lab Assessment

Assessment Task 1: Sampling and Quantization Parameter Variations

- **Objective:** Write a function that samples an image with a given scale factor and quantizes it to a given number of bits, then test it with a scale factor of 0.25 and 2 bits.
- **Code & Output:**

```
def sample_and_quantize(image_path, scale_factor= 0.25, bits= 2):  
    img= Image.open(image_path).convert('L')  
    orig_w, orig_h= img.size  
  
    new_w, new_h= max(1, int(orig_w * scale_factor)), max(1, int(orig_h *  
scale_factor))  
    sampled_img= img.resize((new_w, new_h), Image.Resampling.NEAREST)  
    sampled_restored= sampled_img.resize((orig_w, orig_h), Image.Resampling.NEAREST)  
  
    levels= 2 ** bits  
    arr= np.asarray(sampled_restored, dtype=np.float32)  
    quantized_arr= np.floor(arr / (256.0 / levels)) * (255.0 / (levels - 1))  
    quantized_img= Image.fromarray(quantized_arr.astype(np.uint8))  
  
    return quantized_img  
  
degraded_img= sample_and_quantize('./images/task1v2.png', scale_factor= 0.25, bits= 2)  
degraded_img.show()
```

Input: task1v2.png (grayscale)

Output: degraded_img.show()



With 2 bits there are only 4 gray levels (0, 85, 170, 255), and the 0.25 scale factor gives coarse, blocky edges.

Assessment Task 2: Image Arithmetic and Gray-Scale Set Operations

- **Objective:** Apply image subtraction, constant addition, set difference, symmetric difference, and intersection to two grayscale images X and Y. The images are converted to int32 first to prevent underflow and overflow, and the results are cast back to uint8.
- **Code & Output:**

```
img1= Image.open('./images/task1v1.png').convert('L')
img2= Image.open('./images/task1v2.png').convert('L')

resize_dim= (400, 400)
img1= img1.resize(resize_dim, Image.Resampling.LANCZOS)
img2= img2.resize(resize_dim, Image.Resampling.LANCZOS)

# Convert to float/int32 to prevent underflow/overflow during calculations
X= np.asarray(img1, dtype= np.int32)
Y= np.asarray(img2, dtype= np.int32)
```

X = img1 (task1v1.png)



Y = img2 (task1v2.png)



1. Image subtraction: $|X - Y|$

```
# 1. Subtract two images. Absolute difference  $|X - Y|$  (or clip to 0: np.clip(X - Y, 0, 255))
subtraction= np.abs(X - Y).astype(np.uint8)
Image.fromarray(subtraction).show()
```



Where Y is black the output equals X, and where Y is white it is the negative of X ($255 - X$).

2. Adding a constant (175)

```
# 2. Add constant value (175). Brightens image, clipped to max saturation (255)
addition_const= np.clip(X + 175, 0, 255).astype(np.uint8)
Image.fromarray(addition_const).show()
```



Every pixel with $X \geq 80$ saturates to 255 (about 78% of the pixels), so only the darkest areas remain visible.

3. Set difference: $X \setminus Y = \min(X, 255 - Y)$

```
# 3. Set Difference:  $X \setminus Y$ 
complement_Y= 255 - Y
set_difference= np.minimum(X, complement_Y).astype(np.uint8)
Image.fromarray(set_difference).show()
```



X is kept where Y is dark and suppressed (black) where Y is bright.

4. Symmetric difference: $(X \setminus Y) \cup (Y \setminus X)$

```
# 4. Symmetric Difference:  $(X \setminus Y) \cup (Y \setminus X)$   
complement_X= 255 - X  
diff_X_Y= np.minimum(X, complement_Y)  
diff_Y_X= np.minimum(Y, complement_X)  
symmetric_difference= np.maximum(diff_X_Y, diff_Y_X).astype(np.uint8)  
Image.fromarray(symmetric_difference).show()
```



Combines both one-sided differences to extract the non-overlapping intensity features of the two images.

5. Intersection: $X \cap Y = \min(X, Y)$

```
# 5. Intersection:  $X \cap Y$   
intersection= np.minimum(X, Y).astype(np.uint8)  
Image.fromarray(intersection).show()
```



Keeps the darker of the two intensities at each pixel, so the black silhouette is placed over the other image.